
Concurrencia y Paralelismo

Bloque I: Concurrencia

Mayo 2025

1. Bus [2 puntos]

Un autobús circula por una línea con un cierto número de paradas. La línea es circular, es decir, después de la última parada se vuelve a la primera. Puede suponerse que solo circula un autobús en la línea.

El autobús, al llegar a una parada, avisa a quien esté esperando para subir y bajar, y espera a que los pasajeros suban y bajen del bus. Una vez ha terminado la parada cambia a la siguiente estación.

El código de los threads pasajeros se proporciona como referencia. Implemente la función bus.

```
struct bus_line {
    int current_stop; // Current stop
    int total_stops; // Total number of stops
    pthread_mutex_t m;
    int get_off[STOPS]; // Number of passengers that want leave at that stop
    int load[STOPS]; // Number of passengers that want to enter at that stop
    pthread_cond_t stop[STOPS];
    pthread_cond_t loading;
}

void bus(struct bus_line *bus) {
    ...
}

void passenger(struct bus_line *bus, int from, int to) {
    pthread_mutex_lock(&bus->m);
    if(bus->current_stop != from) {
        bus->load[from]++;
        pthread_cond_wait(&bus->load[from], &bus->m);
        bus->load[from]--;
        if(bus->load[from] == 0) pthread_cond_signal(&bus->loading);
    }
    bus->get_off[to]++;
    pthread_cond_wait(&bus->stop[to], &bus->m);
    bus->get_off[to]--;
    if(bus->get_off[to] == 0) pthread_cond_signal(&bus->loading);
    pthread_mutex_unlock(&bus->m);
}
```

Solución:

```
struct bus_line {
    int current_stop; // Current stop
    int total_stops; // Total number of stops
    pthread_mutex_t m;
    int get_off[STOPS]; // Number of passengers that want leave at that stop
    int load[STOPS]; // Number of passengers that want to enter at that stop
    pthread_cond_t stop[STOPS];
    pthread_cond_t loading;
}

void bus(struct bus_line *bus) {
    pthread_mutex_lock(&bus->m);
    while(true) {
        pthread_mutex_broadcast(bus->get_off[bus->current_stop]);
        pthread_mutex_broadcast(bus->load[bus->current_stop]);
        while(bus->get_off[bus->current_stop] > 0 || bus->load[bus->current_stop] > 0) {
            pthread_cond_wait(&bus->loading, &bus->m);
        }
        bus->current_stop = (bus->current_stop + 1) % bus->total_stops;
    }
    pthread_mutex_unlock(&bus->m);
}

void passenger(struct bus_line *bus, int from, int to) {
    pthread_mutex_lock(&bus->m);
    if(bus->current_stop != from) {
        bus->load[from]++;
        pthread_cond_wait(&bus->load[from], &bus->m);
        bus->load[from]--;
        if(bus->load[from] == 0) pthread_cond_signal(bus->loading);
    }
    bus->get_off[to]++;
    pthread_cond_wait(&bus->stop[to], &bus->m);
    bus->get_off[to]--;
    if(bus->get_off[to] == 0) pthread_cond_signal(bus->loading);
    pthread_mutex_unlock(&bus->m);
}
```

2. Mutex con bloqueo ordenado [1.5 puntos]

Implemente, utilizando los mutex de la librería pthread, un tipo de mutex donde en caso de que tengan que esperar los threads lo bloqueen respetando el orden en el que llegan a la operación lock. Cuando el mutex se libere, solo debería despertarse un thread como máximo.

Para la solución puede usar la cola ya implementada en las funciones `q_init`, `q_insert` y `q_remove`

```
typedef {  
    ...  
} ord_mutex_t;  
  
void q_init(queue q);  
void q_insert(queue q, void *);  
void *q_remove(queue q); // returns NULL if the queue is empty  
  
void ord_mutex_init(ord_mutex_t *m) {  
    ...  
}  
  
void ord_mutex_lock(ord_mutex_t *m) {  
    ...  
}  
  
void ord_mutex_unlock(ord_mutex_t *m) {  
    ...  
}
```

```
int pthread_mutex_init(pthread_mutex_t *m, pthread_mutex_attr_t *attr);  
int pthread_mutex_lock(pthread_mutex_t *m);  
int pthread_mutex_trylock(pthread_mutex_t *m);  
int pthread_mutex_unlock(pthread_mutex_t *m);  
int pthread_cond_init(pthread_cond_t *c, pthread_cond_attr_t *attr);  
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);  
int pthread_cond_signal(pthread_cond_t *cond);  
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Solución:

```
typedef {
    pthread_mutex_t m;
    bool locked;
    queue q;
} ord_mutex_t;

void ord_mutex_init(ord_mutex_t *m) {
    pthread_mutex_init(&m->m, NULL);
    m->locked = false;
    q_init(m->q);
}

void ord_mutex_lock(ord_mutex_t *m) {
    pthread_cond_t c;

    pthread_mutex_lock(&m->m);
    if(m->locked) {
        pthread_cond_init(&c, NULL);
        q_insert(m->q, &c);
        pthread_cond_wait(&c, &m->m);
    }
    m->locked=true;
    pthread_mutex_lock(&m->m);
}

void ord_mutex_unlock(ord_mutex_t *m) {
    pthread_mutex_lock(&m->m);
    if((c = q_remove(m->q)) != NULL) {
        pthread_cond_signal(c);
    } else {
        locked = false;
    }
    pthread_mutex_unlock(&m->m);
}
```

3. Línea de procesos [1.5 puntos]

El siguiente módulo permite crear una secuencia de procesos donde cada uno conoce el PID del siguiente.

```
-module(line).
-export([start/1, size/1]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

size(First) ->
    ...

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
    ...
    end,
    loop(Next).
```

Implemente la función `size/1`, que dado el PID del primer proceso devuelva el número de procesos de la línea. Las funciones `start/1` e `init/1` no pueden modificarse.
Solución:

```
-module(line).
-export([start/1, init/1, size/1]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

size(First) ->
    First ! {get_size, self(), 0},
    receive
        {get_size_reply, N} ->
            N
    end.

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
        {get_size, From, N} ->
            case Next of
                none ->
                    From ! {get_size_reply, N + 1};
                _ ->
                    Next ! {get_size, From, N + 1}
            end
    end,
    loop(Next).
```
